Pattern Recognition 83 (2018) 161–173

Contents lists available at ScienceDirect

Pattern Recognition

journal homepage: www.elsevier.com/locate/patcog

Discriminatively boosted image clustering with fully convolutional auto-encoders

Fengfu Li[a,b], Hong Qiao[c,d,e], Bo Zhang[a,b,*]

[a] Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing 100190, China
[b] School of Mathematical Sciences, University of Chinese Academy of Sciences, Beijing 100049,China
[c] Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China
[d] University of Chinese Academy of Sciences, Beijing 100049, China
[e] CAS Centre for Excellence in Brain Science and Intelligence Technology, Shanghai 200031, China

ARTICLE INFO

Article history:
Received 23 March 2017
Revised 10 January 2018
Accepted 20 May 2018
Available online 21 May 2018

Keywords:
Image clustering
Fully convolutional auto-encoder
Representation learning
Discriminatively boosted clustering

ABSTRACT

Traditional image clustering methods take a two-step approach, feature learning and clustering, sequentially. However, recent research results demonstrated that combining the separated phases in a unified framework and training them jointly can achieve a better performance. In this paper, we first introduce fully convolutional auto-encoders for image feature learning and then propose a unified clustering framework to learn image representations and cluster centers jointly based on a fully convolutional auto-encoder and soft *k*-means scores. At initial stages of the learning procedure, the representations extracted from the auto-encoder may not be very discriminative for latter clustering. We address this issue by adopting a boosted discriminative distribution, where high score assignments are highlighted and low score ones are de-emphasized. With the gradually boosted discrimination, clustering assignment scores are discriminated and cluster purities are enlarged. Experiments on several vision benchmark datasets show that our methods can achieve a state-of-the-art performance.

1. Introduction

Clustering methods are very important techniques for exploratory data analysis with wide applications ranging from data mining [1,2], image segmentation [3–5] and so on. Their aim is to partition data points into clusters so that data in the same cluster are similar to each other while data in different clusters are dissimilar. Approaches to achieve this aim include partitional methods such as fuzzy c-means [6], hierarchical methods like agglomerative clustering and divisive clustering, methods based on density estimation such as DBSCAN [7], and recent methods based on finding density peaks such as CFSFDP [8].

Image clustering [9,10] is a special case of clustering analysis that seeks to find compact, object-level models from many unlabeled images. Its applications include automatic visual concept discovery, content-based image retrieval, image annotation, and so on. However, image clustering is a hard task mainly owning to the following two reasons: 1) images often are of high dimensionality, which will significantly affect the performance of clustering methods such as *k*-means [11], and 2) objects in images usually have two-dimensional or three-dimensional local structures which should not be ignored when exploring the local structure information of the images.

To address these issues, many representation learning methods have been proposed for image feature extractions as a preprocessing step. Traditionally, various hand-crafted features such as SIFT [12], LBP [13], NMF [14], and CW-SSIM similarity [15] have been used to encode the visual information. Recently, many approaches have been proposed to combine clustering methods with deep neural networks (DNN), which have shown a remarkable performance improvement over hand-crafted features [16,23]. Roughly speaking, these methods can be categorized into two groups: 1) sequential methods that apply clustering on the learned DNN representations, and 2) unified approaches that jointly optimize the deep representation learning and clustering objectives.

In the first group, a kind of deep (convolutional) neural networks is first trained in an unsupervised manner to approximate the non-linear feature embedding from the raw image space to the embedded feature space (usually being low-dimensional) [17]. And then, either *k*-means or spectral clustering or agglomerative clustering can be applied to partition the feature space. However, since the feature learning and clustering are separated from each other, the learned DNN features may not be reliable for clustering.

* Corresponding author at: Institute of Applied Mathematics, 55 Zhongguancun Donglu, Beijing 100190, China.
E-mail address: b.zhang@amt.ac.cn (B. Zhang).

https://doi.org/10.1016/j.patcog.2018.05.019
0031-3203/

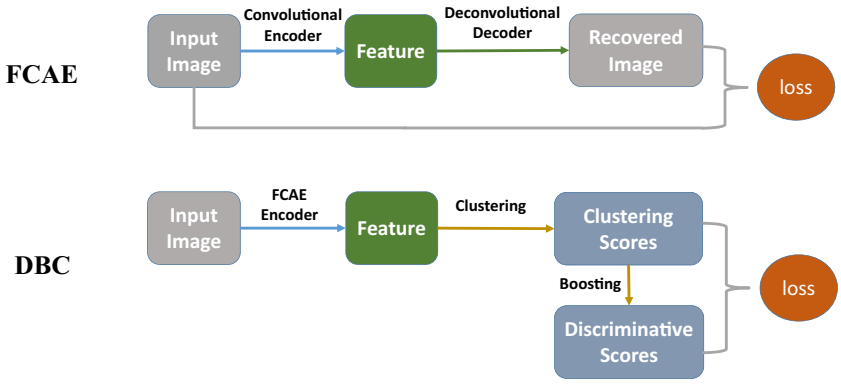


Fig. 1. Diagram of the proposed fully convolutional auto-encoders and discriminatively boosted clustering.

There are a few recent methods in the second group which take the separation issues into consideration. In [18], the authors proposed deep embedded clustering that simultaneously learns feature representations with stacked auto-encoders and cluster assignments with soft k-means by minimizing a joint loss function. In [19], joint unsupervised learning was proposed to learn deep convolutional representations and agglomerative clustering jointly using a recurrent framework. In [20], the authors proposed an infinite ensemble clustering framework that integrates deep representation learning and ensemble clustering. The key insight behind these approaches is that good representations are beneficial for clustering and conversely clustering results can provide supervisory signals for representation learning. Thus, two factors, designing a proper representation learning model and designing a suitable unified learning objective will greatly affect the performance of these kind of methods.

In this paper, we follow recent advances to propose a unified clustering method named discriminatively boosted clustering (DBC) for image analysis based on fully convolutional auto-encoders (FCAE). See Fig. 1 for the diagram of the them. We first introduce a fully convolutional encoder-decoder network for fast and coarse image feature extraction. We then discard the decoder part and add a soft k-means model on top of the encoder to make a unified clustering model. The model is jointly trained with gradually boosted discrimination where high score assignments are highlighted and low score ones are de-emphasized. The our main contributions are summarized as follows:

- We propose a fully convolutional auto-encoder (FCAE) for image feature learning. The FCAE is composed of convolution-type layers (convolution and de-convolution layers) and pool-type layers (pooling and un-pooling layers). By adding batch normalization (BN) layers to each of the convolution-type layers, we can train the FCAE in an end-to-end way. This avoids the tedious and time-consuming layer-wise pre-training stage adopted in the traditional stacked (convolutional) auto-encoders. To the best of our knowledge, this is the first attempt to learn a deep auto-encoder in an end-to-end manner.
- We propose a discriminatively boosted clustering (DBC) framework based on the learned FCAE and an additional soft k-means model. We train the DBC model in a self-paced learning procedure, where deep representations of raw images and cluster assignments are jointly learned. This overcomes the separation issue of the traditional clustering methods that use features directly learned from auto-encoders.
- We show that the FCAE can learn better features for clustering than raw images on several image datasets include MNIST, USPS, COIL-20 and COIL-100. Besides, with discriminatively boosted learning, the FCAE based DBC can outperform several state-of-the-art analogous methods in terms of k-means and deep auto-encoder based clustering.

The remaining part of this paper is organized as follows. Some related work including stacked (convolutional) auto-encoders, deconvolutional neural networks, and joint feature learning and clustering are briefly reviewed in Section 2. Detailed descriptions of the proposed FCAE and DBC are presented in Section 3. Experimental results on several real datasets are given in Section 4 to validate the proposed methods. Conclusions and future works are discussed in Section 5.

2. Related work

Stacked auto-encoders have been studied in the past years for unsupervised deep feature extraction and nonlinear dimensionality reduction. In [21], Restricted Boltzmann Machines (RBMs) were stacked layer-by-layer to form a deep auto-encoder for dimensionality reduction. In [28], Stacked Autoassociators Network was proposed to learn hierarchical features. Vincent et al. [22] introduced stacked denoising auto-encoders (SDAEs), which corrupts the input with random noise to make the algorithm more robust to variation than ordinary SAEs and gets superior classification performance. Though the above SAEs can learn hierarchical features from raw data, they are not suited to deal with structural image data. Masci et al. [25] and Lee et al. [26] proposed convolutional extensions of the SAEs (namely stacked convolutional auto-encoders, or SCAEs) to address the issue. They take 2-D or 3-D image as the input and adopt convolution operation along with pooling operation to train the SCAEs. Compared with the SAEs, the SCAEs can reserve more structural information. Though the SAEs and SCAEs are powerful unsupervised feature extraction methods, their training usually contain a tedious two-stage procedure [21,24]: one is layer-wise pre-training and the other is overall fine-tuning. One of the significant drawbacks of this learning procedure is that the layer-wise pre-training is time-consuming and tedious, especially when the base layer is a RBM rather than an ordinary auto-encoder, or when the overall network is very deep.

Recently, there is an attempt to discard the layer-wise pre-training procedure and train a deep auto-encoder type network in an end-to-end way. In [29], a deep deconvolution network was learned for image segmentation. The input of the architecture is an image and the output is a segmentation mask. The network achieves the state-of-the-art performance compared with analogous methods thanks to three factors: 1) introducing a deconvolution layer and a unpooling layer to recover the original image size of the segmentation mask, 2) applying the batch normalization strategy [30] to each convolution layer and each deconvolution layer to reduce the internal covariate shifts, which not only

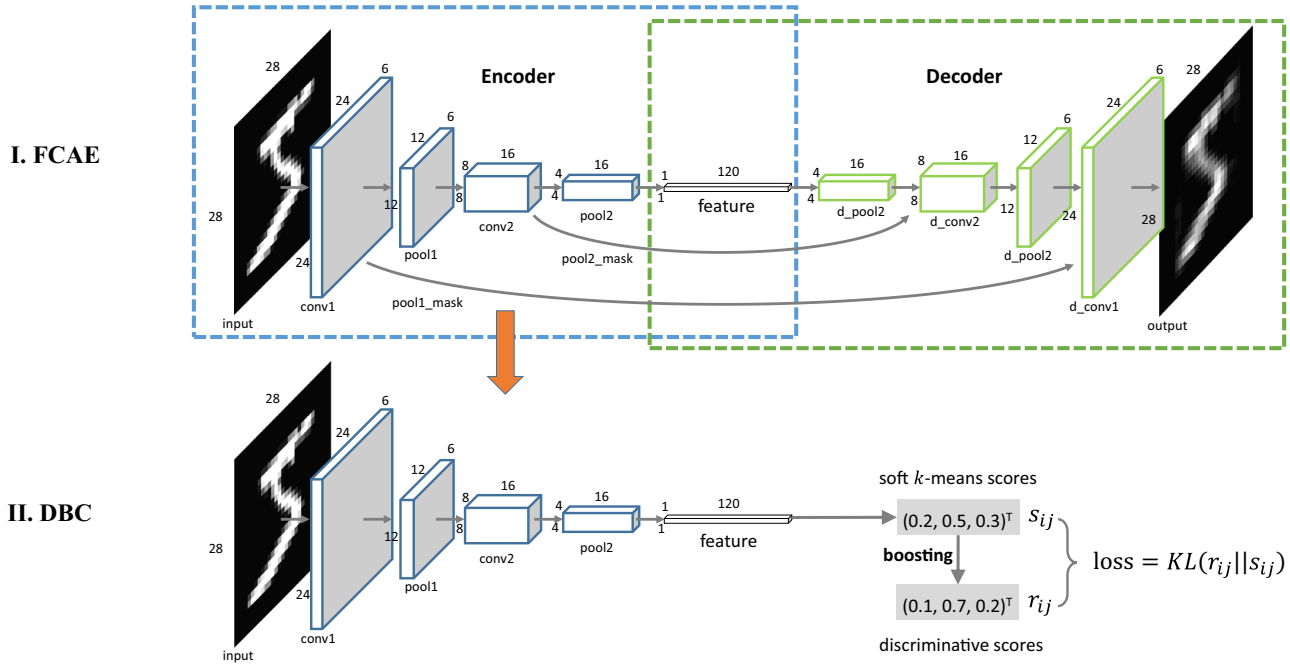


Fig. 2. An example architecture of the unified image clustering framework. Part I is a 4-layer fully convolutional auto-encoder (FCAE), and Part II is a discriminatively boosted clustering (DBC) framework based on the FCAE and the k-means clustering method.

makes an end-to-end training procedure possible but also speeds up the process, and 3) adopting a pre-trained encoder on large-scale datasets. The success of the architecture motivates us that it is possible to design an end-to-end training procedure for fully convolutional auto-encoders.

Clustering has also been studied in the past years based on independent features extracted from auto-encoders. In [32], the authors made a first attempt to develop auto-encoder for clustering. They proposed to use auto-encoder network [17] for feature extraction and then clustering based on the learned features. As a result, the learned features are more stable and compact than raw features, and are thus more suitable for clustering. In [31], deep embedding network (DEN) was proposed to adopt multi-layer Gaussian RBMs as feature learner with locality-preserving constraint and group sparsity constraint. Thanks to the constrains, the method can learn clustering-oriented representations. In [27], GraphEncoder was proposed to employ deep sparse auto-encoders for graph clustering and achieved better performance than spectral methods. Though the above methods have achieved some success, they have a significant drawback: the feature learning procedure and the clustering procedure are separated. To address the issue, attempts have been made to combine the auto-encoders and clustering in a unified framework recently. In [18], the authors proposed Deep Embedded Clustering (DEC) that learns deep representations and cluster assignments jointly. DEC uses a deep stacked auto-encoder to initialize the feature extraction model and a Kullback–Leibler divergence loss to fine-tune the unified model. However, it is not suitable for dealing with high-dimensional images due to the use of stacked auto-encoders rather than convolutional ones. This motivates us to design a unified clustering framework based on convolutional auto-encoders for image clustering.

3. Proposed methods

In this section, we propose a unified image clustering framework with fully convolutional auto-encoders and a soft k-means clustering model. Fig. 2 shows an example of the overall framework on the MNIST data set. The framework contains two parts: part I is a fully convolutional auto-encoder (FCAE) for fast and coarse image feature extraction, and part II is a discriminatively boosted clustering (DBC) method which is composed of a fully convolutional encoder and a soft k-means categorizer. The DBC takes an image as input and exports soft assignments as output. It can be jointly trained with a discriminatively boosted distribution assumption, which makes the learned deep representations more suitable for the top categorizer. The learning procedure is self-paced, where easiest instances are first focused and more complex objects are expanded progressively. In the following subsections, we will explain the detailed implementation of the method.

3.1. Fully convolutional auto-encoder for image feature extraction

Traditional deep convolutional auto-encoders adopt a greedy layer-wise training procedure for feature transformations. This could be tedious and time-consuming when dealing with very deep neural networks. To address this issue, we propose a fully convolutional auto-encoder architecture which can be trained in an end-to-end manner. Part I of Fig. 2 shows an example of FCAE on the MNIST dataset. It has the following features:

- **Fully convolutional:** As pointed out in [25], the max-pooling layers are very crucial for learning biologically plausible features in the convolutional architectures. Thus, we adopt convolution layers along with max-pooling layers to make a fully convolutional encoder (FCE). Since the down-sampling operations in the FCE reduce the size of the output feature maps, we use an unpooling layer introduced in [29] to recover the feature maps. As a result, the unpooling layers along with deconvolution layers (see [29]) are adopted to make a fully convolutional decoder (FCD).
- **Symmetric:** The overall architecture is symmetric around the feature layer. In practice, it is suggested to design layers of an odd number. Otherwise, it will be ambiguous to define the feature layer. Besides, fully connected layers (dense layers) should be avoided in the architecture since they destroy the local structure of the feature layer.

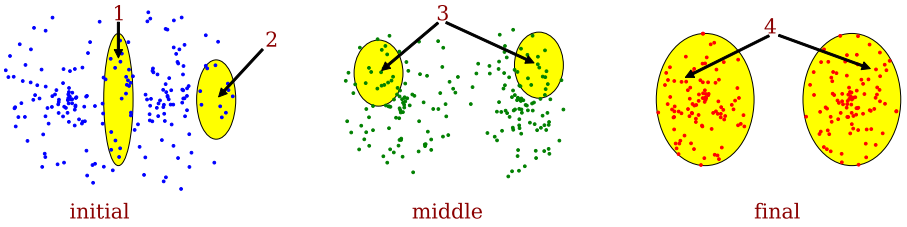


Fig. 3. Learning procedure of DBC.

Normalized: The depth of the whole network grows in log-magnitude as the input image size increases. This could make the network very deep if the original image has a very large width or height. To overcome this problem, we adopt the batch normalization (BN) [30] strategy for reducing the internal covariate shift and speeding up the training. The BN operation is performed after each convolutional layer and each deconvolutional layer except for the last output layer. As pointed out in [29], BN is critical to optimize the fully convolutional neural networks.

FCAE utilizes the two-dimensional local structure of the input images and reduces the redundancy in parameters compared with stacked auto-encoders (SAEs). Besides, FCAE differs from conventional SAEs as its weights are shared among all locations within each feature map and thus preserves the spatial locality.

3.2. Discriminatively boosted clustering

Once FCAE has been trained, we can extract features with the encoder part to serve as the input of a categorizer. This strategy is used in many clustering methods based on auto-encoders, such as GraphEncoder [27], deep embedding networks [31], and auto-encoder based clustering [32]. These approaches treat the auto-encoder as a preprocessing step which is separately designed from the latter clustering step. However, the representations learned in this way could be amphibolous for clustering, and the clusters may be unclear (see the initial stage in Fig. 3).

To address this issue, we propose a self-paced approach to make feature learning and clustering in a unified framework (see Part II in Fig. 2). We throw away the decoder of the FACE and add a soft k-means model on top of the feature layer. To train the unified model, we trust easier samples first and then gradually utilize new samples with the increasing complexity. Here, an *easier* sample (see the regions labeled 2, 3 and 4 in Fig. 3) is much certain to belong to a specific cluster, and a *harder* sample (see the region 1 in Fig. 3) is very likely to be categorized to multiple clusters. Fig. 3 describes the difference between these samples at a different learning stage of DBC.

There are three challenging questions in the learning problem of DBC which will be answered in the following subsections:

1. How to choose a proper criterion to determine the easiness or hardness of a sample?
2. How to transform harder samples into easier ones?
3. How to learn from easier samples?

3.2.1. Easiness measurement with the soft k-means scores

We follow DEC [18] to adopt the t-distribution-based soft assignment to measure the easiness of a sample. The t-distribution is investigated in [33] to deal with the crowding problem of low-dimensional data distributions. Under the t-distribution kernel, the soft score (or similarity) between the feature z_i ($i \in 1, 2, \ldots, m$) and the cluster center μ_j ($j \in 1, 2, \ldots, k$) is

$$s_{ij} \propto \left(1 + \frac{||z_i - \mu_j||^2}{v}\right)^{-\frac{v+1}{2}} \quad (1)$$

$$\text{s.t.} \sum_{j=1}^{k} s_{ij} = 1$$

Here, v is the degree of freedom of the t-distribution and set to be 1 in practice. The most important reason for choosing the t-distribution kernel is that it has a longer tail than the famous heat kernel (or the Gaussian-distribution kernel). Thus, we do not need to pay much attention to the parameter estimation (see [33]), which is a hard task in unsupervised learning.

3.2.2. Boosting easiness with discriminative target distribution

We transform the harder examples to the easier ones by boosting the higher score assignments and, meanwhile, bring down those with lower scores. This can be achieved by constructing an underlying target distribution r_{ij} from s_{ij} as follows:

$$r_{ij} \propto s_{ij}^{\alpha}, \ \alpha > 1 \quad (2)$$

$$\text{s.t.} \sum_{j=1}^{k} r_{ij} = 1$$

Suppose we can ideally learn from the soft scores (denoted as S) to the assumptive distribution (denoted as R) each time. Then we can generate a learning chain as follows:

$$S^{(0)} \to R^{(0)} = S^{(1)} \to R^{(1)} = S^{(2)} \to \ldots$$

The following two properties can be observed from the chain:

Property 1. *If $s_{ij}^{(0)} = s_{ij'}^{(0)}$ for any j and j', then $s_{ij}^{(t)} \equiv 1/k$ for all j and all time step t.*

Proof. Under the condition, and by (2) we can deduce that $r_{ij}^{(0)} \equiv r_{ij'}^{(0)}$. By the chain this is equivalent to the fact that $s_{ij}^{(1)} \equiv s_{ij'}^{(1)}$. Thus, the conclusion $s_{ij}^{(t)} \equiv s_{ij'}^{(t)}$ follows recursively for all t. □

Property 2. *If there exists an l such that $s_{il}^{(0)} > \max_{j \neq l} s_{ij}^{(0)}$, then*

$$\lim_{t \to \infty} s_{ij}^{(t)} = \begin{cases} 1 & \text{if } j = l \\ 0 & \text{if } j \neq l \end{cases}$$

Proof. By (2) we have

$$\frac{s_{ij}^{(t)}}{s_{il}^{(t)}} = \left[\frac{s_{ij}^{(t-1)}}{s_{il}^{(t-1)}}\right]^{\alpha} = \left[\frac{s_{ij}^{(t-2)}}{s_{il}^{(t-2)}}\right]^{\alpha^2} = \ldots = \left[\frac{s_{ij}^{(0)}}{s_{il}^{(0)}}\right]^{\alpha^t}.$$

By the assumption $s_{il}^{(0)} > \max_{j \neq l} s_{ij}^{(0)}$, it is seen that $s_{ij}^{(0)}/s_{il}^{(0)} < 1$ for any $j \neq l$. On the other hand, since $\alpha > 1$, we have $\lim_{t \to \infty} \alpha^t = \infty$. Thus,

$$\lim_{t \to \infty} \frac{s_{ij}^{(t)}}{s_{il}^{(t)}} = \lim_{\alpha^t \to \infty} \left[\frac{s_{ij}^{(0)}}{s_{il}^{(0)}}\right]^{\alpha^t} = 0, \quad \forall j \neq l.$$

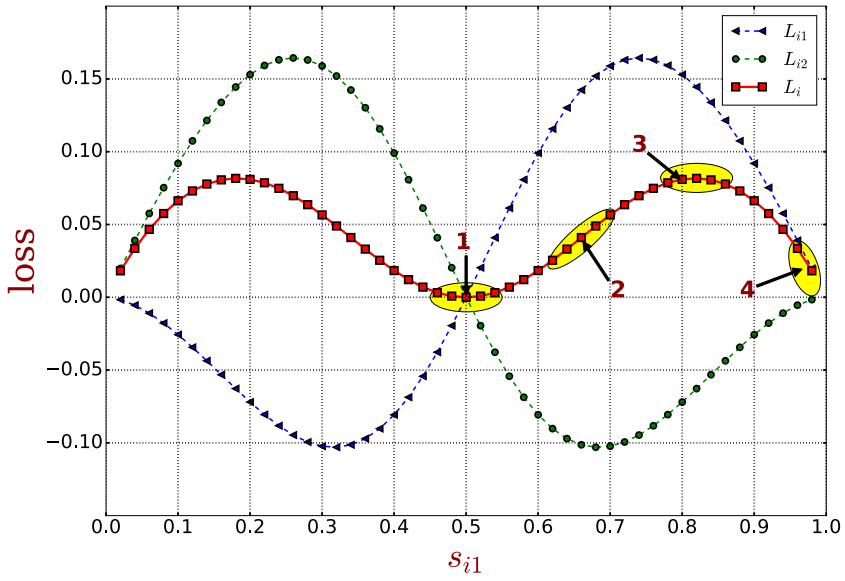


Fig. 4. KL divergence loss with respect to the soft scores assigned to the first cluster. Here we assume that there are 2 clusters, so 0.5 is a random guess probability.

Since $s_{il}^{(t)}$ is finite, we have $\lim_{t\to\infty} s_{ij}^{(t)} = 0, \forall j \neq l$. Finally, with the constrains $\sum_{j=1}^{k} s_{ij}^{(t)} = 1$, we obtain

$$\lim_{t\to\infty} s_{il}^{(t)} = 1 - \sum_{j\neq l} \lim_{t\to\infty} s_{ij}^{(t)} = 1.$$

□

Property 1 tells us that the *hardest* sample (which has the equal probability to be assigned to different clusters) would always be the hardest one. However, in practical applications, there can hardly exist such examples. Property 2 shows that the initial non-discriminative samples could be boosted gradually to be definitely discriminative. As a result, we get the desired features for k-means clustering.

Note that the boosting factor α controls the speed of the learning process. A larger α can make the learning process more quickly than smaller ones. However, it may boost some falsely categorized samples too quickly at initial stages and thus makes their features irrecoverable at later stages.

Besides, it can be helpful to balance the data distribution at different learning stages. In [18], the authors proposed to normalize the boosted assignments to prevent large clusters from distorting the hidden feature space. This problem can be overcome by dividing a normalization factor $n_j = \sum_{i=1}^{m} r_{ij}$ for each of the r_{ij}.

3.2.3. Learning with the Kullback–Leibler divergence loss

In the last subsection, it was assumed that we could learn from s_{ij} to the boosted target distribution r_{ij}. This aim can be achieved with a joint Kullback–Leibler (KL) divergence loss, that is,

$$(\theta^*, \mu^*) = \arg\min_{\theta,\ \mu} L = \mathrm{KL}(R||S) = \sum_{i=1}^{m}\sum_{j=1}^{k} r_{ij} \log \frac{r_{ij}}{s_{ij}}. \tag{3}$$

Fig. 4 gives an example of the joint loss when $k = 2$, where $L_{ij} = r_{ij}\log(r_{ij}/s_{ij})$ is the loss generated by the sample x_i with respect to the jth cluster ($j = 1$ or 2). Regions marked in Fig. 4 roughly correspond to the regions marked in Fig. 3. Intuitively, the loss has the following main features:

- For an ambiguous (or hard) sample (i.e., $s_{ij} \approx s_{il}$, $\forall j$, l), its loss $L_i = \sum_{j=1}^{k} r_{ij}\log(r_{ij}/s_{ij}) \approx 0$ according to Property 1. Therefore, it will not be seriously treated in the learning process. (Region 1)
- For a good categorized sample (i.e., there exists an l such that $1 \gg s_{il} > \max_{j\neq l} s_{ij}$), its loss will be much greater than zero, and thus it will be treated more seriously. (Regions 2 and 3)
- For a definitely well categorized sample (i.e., there exists an l such that $1 \approx s_{il} \gg \max_{j\neq l} s_{ij}$), its loss will be near zero. This means that its features do not need to be changed much more. (Region 4)

By (1)–(3), the gradients of the KL divergence loss w.r.t. z_i and μ_j can be deduced as follows:

$$\frac{\partial L}{\partial z_i} = \frac{1+\nu}{\nu}\sum_{j=1}^{k}(r_{ij} - s_{ij})\frac{z_i - \mu_j}{1 + ||z_i - \mu_j||^2/\nu} \tag{4}$$

and

$$\frac{\partial L}{\partial \mu_j} = \frac{1+\nu}{\nu}\sum_{i=1}^{m}(r_{ij} - s_{ij})\frac{\mu_j - z_i}{1 + ||\mu_j - z_i||^2/\nu} \tag{5}$$

The derivation of (4) and (5) can be found in the appendix.

3.2.4. Training algorithm

In this section, we summarize the overall training procedure of the proposed method in Algorithms 1 and 2. They implement the framework showed in Fig. 2. Here T is the maximum learning epochs, B is the maximum updating iterations in each epoch and

Algorithm 1 Discriminatively boosted clustering (DBC).

Require: X, T, B, m_b, α, k
Ensure: θ and μ
Stage I: Train a FCAE and clustering with its features
1: Train a deep fully convolutional auto-encoder

$$x_i \xrightarrow{\theta_e} z_i(\text{features}) \xrightarrow{\theta_d} \longrightarrow \hat{x}_i \tag{M1}$$

with the Euclidian loss

$$(\theta_e^*, \theta_d^*) = \arg\min_{\theta_e,\theta_d} \sum_{i=1}^{m} ||x_i - \hat{x}_i||_2^2 = \sum_{i=1}^{m} ||x_i - g_{\theta_d}(f_{\theta_e}(x_i))||_2^2$$

by using the traditional error back-propagation algorithm.
2: Extract features: $Z \leftarrow f_{\theta_e^*}(X)$
3: Clustering with the features: $\mu_z \leftarrow k$-means centers

m_b is the mini-batch size. The encoder part of FCAE is f: $x \to z$, which is parameterized by θ_e and the decoder part of FCAE is g: $z \to \hat{x}$, which is parameterized by θ_d.

Algorithm 2 DBC (Continued).

Stage II: Jointly learn the FCE and cluster centers

Construct a unified clustering model with encoder parameters θ and cluster centers μ

$$x_i \xrightarrow{\theta} z_i \xrightarrow{\mu_j} s_{ij} \qquad (M2)$$

Initialization: $\theta \leftarrow \theta_e^*$, $\mu \leftarrow \mu_z$

for t = 1 to T **do**

Forward propagate (M2) and update the *soft* assignments

$$s_{ij} \leftarrow \frac{(1+||z_i-\mu_j||^2)^{-1}}{\sum_{j=1}^{k}(1+||z_i-\mu_j||^2)^{-1}}, \text{where } z_i = f_\theta(x_i).$$

Update the target distribution

$$r_{ij} \leftarrow \frac{s_{ij}^{\alpha}/n_j}{\sum_{j=1}^{k} s_{ij}^{\alpha}/n_j}, \text{where } n_j = \sum_{i=1}^{m} s_{ij}^{\alpha}.$$

for b = 1 to B **do**

Forward propagate (M2) with a mini-batch of m_b samples.

Backward propagate (M2) from (4) and (5)to get $\partial L/\partial\theta$ and $\partial L/\partial\mu$.

Update θ and μ with the gradients.

end for

Stop if *hard* assignments remain unchanged.

end for

3.2.5. Time complexity analysis

We analyze the training and running time complexity of the proposed FCAE and DBC. In the following analysis, we assume that the depth of the encoder is n, the feature map size is $v_i \times v_i$, the number of the feature maps in the ith layer is k_i and the kernel size of the filters is $w_i \times w_i$. Here, the length and width of the feature maps are assumed to be the same; the same assumption applies to the kernels. With the notations and assumptions, the forward and backward time complexity is $O(w_i^2 v_i^2 k_i k_{i+1})$ with respect to the ith layer. Thus, the total training time of FCAE is $O(Tm\sum_{i=0}^{l-1} w_i^2 v_i^2 k_i k_{i+1} + kmv_l^2)$, where T is the total training epochs, m is the number of data samples and m_b is the mini-batch size. For DBC, the time complexity of the clustering assignment stage is $O(mv_l^2 k + Bm_b v_l^2 k)$ in one epoch. Thus, the total training time complexity of DBC is $O(T(m+Bm_b)(\sum_{i=0}^{l-1} w_i^2 v_i^2 k_i k_{i+1} + kv_l^2))$. In real applications, $O(Bm_b)$ is in the same magnitude as $O(m)$, and kv_l^2 is much smaller than $\sum_{i=0}^{l-1} w_i^2 v_i^2 k_i k_{i+1}$. As a result, FCAE and DBC have the same amount of training time $O(Tm\sum_{i=0}^{l-1} w_i^2 v_i^2 k_i k_{i+1})$, which is the same as the ordinary convolutional auto-encoders.

4. Experiments

In this section, we present experimental results on several real datasets to evaluate the proposed methods by comparing with several state-of-the-art methods. To this end, we first introduce several evaluation benchmarks and then present visualization results of the inner features, the learned FCAE weights, the frequency hist of soft assignments during the learning process and the features embedded in a low-dimensional space. We will also give some ablation studies with respect to the boosting factor α, the normalization factor n_j and the FCAE initializations.

Table 1
Datasets used in our experiments.

Dataset	#Samples	#Categories	Image size	#Channels
MNIST	70,000	10	28 × 28	1
USPS	11,000	10	16 × 16	1
COIL-20	1440	20	128 × 128	1
COIL-100	7200	100	128 × 128	3

4.1. Evaluation benchmarks

Datasets. We evaluate the proposed FCAE and DBC methods on two hand-written digit image datasets (MNIST[1] and USPS[2]) and two multi-view object image datasets (COIL-20[3] and COIL-100[4]). The size of the datasets, the number of categories, the image sizes and the number of channels are summarized in Table 1.

Evaluation metrics. Two standard metrics are used to evaluate the experiment results explained as follows.

- Accuracy (ACC) [18]. Given the ground truth labels $\{c_i|1 \le i \le m\}$ and the predicted assignments $\{\hat{c}_i|1 \le i \le m\}$, ACC measures the average accuracy:

$$\mathrm{ACC}(\hat{c}, c) = \max_g \frac{1}{m}\sum_{i=1}^{m} \mathbf{1}\{c_i = g(\hat{c}_i)\}$$

  where g ranges over all possible one-to-one mappings between the labels of the predicted clusters and the ground truth labels. The optimal mapping can be efficiently computed using the Hungarian algorithm [34].
- Normalized mutual information (NMI) [35]. From the information theory point of view NMI can be interpreted as

$$\mathrm{NMI}(\hat{c}, c) = \frac{\mathrm{MI}(\hat{c}, c)}{\max(\mathrm{H}(\hat{c}), \mathrm{H}(c))}$$

  where $\mathrm{H}(c)$ is the entropy of c and $\mathrm{NMI}(\hat{c}, c)$ is the mutual information of $\hat{c}$ and c.

Network architectures. Table 2 shows the network architecture of the encoder parts with respect to different datasets. The decoder parts are totally reversed by the encoder parts. We use max-pooling in all the experiments. The size of all the feature layers is 1×1. No padding is used in the convolutional layers except for the USPS dataset whose padding size is 1.

Comparing methods. To validate the effectiveness of FCAE and DBC, we compare them with the following state-of-the-art methods in terms of the k-means and deep auto-encoders based clustering. **KMS** is the baseline method that applies the k-means algorithm on raw images. **AEC** [32] is a variant of DAE-KMS that simultaneously optimizes the data reconstruction error and representation compactness. **IEC** [20] incorporates the deep representation learning and ensemble clustering. **DEN** [31] learns the clustering-oriented representations by utilizing deep auto-encoders and manifold constraints. **DAE-KMS** [18] uses deep auto-encoders for feature extraction and then applies k-means for later clustering. **DEC** [18] simultaneously learns the feature representations and cluster centers using deep auto-encoders and soft k-means, respectively. **SCAE-KMS** uses stacked convolutional auto-encoders [25] for feature extraction, and then applies k-means clustering on the learned features. **FCAE-KMS** (our algorithm) adopts FCAE for feature extraction and applies k-means for the latter clustering (see Algorithm 1). **DBC** (our algorithm) uses Algorithms 1 and

[1] http://yann.lecun.com/exdb/mnist/ .
[2] http://www.cs.nyu.edu/~roweis/data.html .
[3] http://www.cs.columbia.edu/CAVE/software/softlib/coil-20.php .
[4] http://www.cs.columbia.edu/CAVE/software/softlib/coil-100.php .

Table 2
Detailed configuration of the network architecture of the convolutional encoder. The first rows are the filter sizes of the corresponding layer (filter size or pooling size, #filters). The second rows are the output sizes (feature map size, #channels).

Datasets	conv1	pool1	conv2	pool2	conv3	pool3	conv4	pool4	Features
MNIST	5, 6	2, –	5, 16	2, –	–	–	–	–	4, 120
	24, 6	12, 6	8, 16	4, 16	–	–	–	–	1, 120
USPS	3, 20	2, –	3, 20	2, –	–	–	–	–	4, 160
	16, 20	8, 20	8, 20	4, 20	–	–	–	–	1, 160
COIL	9, 20	2, –	5, 20	2, –	5, 20	2, –	5,40	2, –	4, 320
	120, 20	60, 20	56, 20	28, 20	24, 20	12, 20	8, 40	4, 40	1, 320

Table 3
Clustering performance on MNIST.

Metric	KMS	AEC	IEC	DAE-KMS	DEC	SCAE-KMS	FCAE-KMS	DBC
ACC	0.535	0.760	0.609	0.807	0.841	0.823	0.794	**0.964**
NMI	0.531	0.669	0.542	0.702	0.721	0.738	0.698	**0.917**

Table 4
Clustering performance on USPS.

Metric	KMS	AEC	IEC	DAE-KMS	DEC	SCAE-KMS	FCAE-KMS	DBC
ACC	0.565	0.715	**0.767**	0.673	0.712	0.681	0.667	0.743
NMI	0.523	0.651	0.641	0.642	0.675	0.660	0.645	**0.724**

Table 5
Clustering performance on COIL-20.

Metric	KMS	DEN	DAE-KMS	DEC	SCAE-KMS	FCAE-KMS	DBC
ACC	0.592	0.725	0.698	0.712	0.723	0.787	**0.793**
NMI	0.767	0.870	0.823	0.836	0.843	0.882	**0.895**

Table 6
Clustering performance on COIL-100.

Metric	KMS	IEC	DAE-KMS	DEC	SCAE-KMS	FCAE-KMS	DBC
ACC	0.506	0.546	0.525	0.534	0.703	0.766	**0.775**
NMI	0.772	0.787	0.761	0.783	0.817	0.897	**0.905**

2 for training a unified clustering method. Results of AEC, IEC and DEN are extracted from the literature. In contrast, the results of KMS, DAE-KMS and DEC are generated with the released codes.[5] For SCAE-KMS, on one hand we use the same convolutional architectures as FCAE-KMS for fair comparison; on the other hand, it differs from FCAE-KMS with two points: 1) the training procedure of SCAE is the same as the ordinary auto-encoders, and 2) SCAE does not use batch-normalization strategy which is only used in FCAE.

Results and analysis. Tables 3–6 show the benchmark results on datasets listed in Table 1. The best results are highlighted in bold font. As can be seen from the tables, four observations are very clear: 1) auto-encoders based feature learning plays a key role for clustering. On all the datasets, the *k*-means method performs worst on raw inputs. However, with the learned features, its clustering performance can be improved by large margin. 2) convolutional architectures are more effective than non-convolutional methods for image feature extraction. On the hand written digit sets, clustering performance based on DAE and SCAE/FCAE are very close; however, when dealing with high dimensional COIL sets, clustering performance based on DAE has a big gap compared with clustering based on SCAE/FCAE. 3) FCAE, trained in an end-to-end manner, can have comparable or even more better performance compared with greedy layer-wisely pre-trained SCAE. In shallow architectures (the hand written digits), the performances of FCAE-KMS are slightly worse than that of SCAE-KMS. However, in deep architectures (the COIL sets), FCAE-KMS performs much better than SCAE-KMS. It suggests that the batch normalization operation is very necessary for deep architectures. 4) the discriminatively boosting procedure helps to get better clustering results. This conclusion can be conducted from the comparative experimental results of DEC versus DAE-KMS and DBC versus FCAE-KMS. Thus, by a combination of convolutional architecture, batch-normalization for deep neural networks, and discriminatively boosted learning procedure, the proposed DBC outperforms most of the opponents in terms of both clustering accuracy and normalized mutual information.

Despite DBC has many advantages over other methods, its performance can be heavily affected by the cluster sparsity. From the benchmark results we observed that the performance of DBC is significantly better than that of FCAE-KMS on the digit sets, while the performance of both methods is very similar on the COIL sets. This is because the number of samples is much larger than the number of categories in the hand-written digits datasets. This results in the fact that the distribution of the FCAE features may be closely related and that a lot of ambiguous samples may occur. As a result, the discriminatively boosting procedure makes sense

[5] https://github.com/piiswrong/dec .

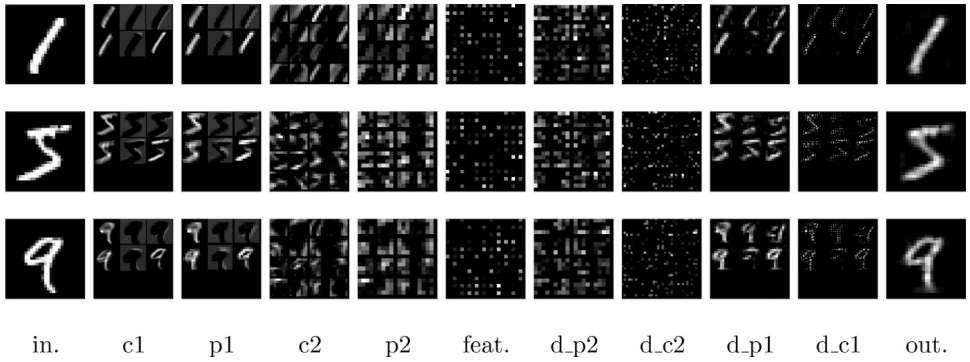


Fig. 5. Visualization of the inner activations with respect to digits 1, 5 and 9.

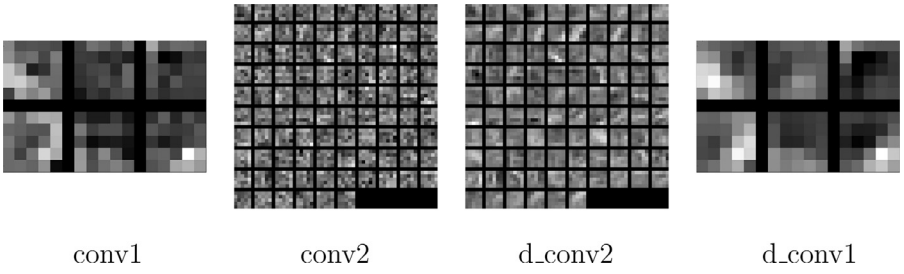


Fig. 6. Visualization of the learned FCAE filters.

on these datasets. Therefore, DBC will perform much better than FCAE-KMS. On the COIL sets, DBC only takes a small advantage of the discriminatively boosting procedure since the FCAE features are very sparse. In other words, there are very few ambiguous samples whose easiness needs to be boosted. As a result, the performance of FCAE-KMS and DBC is very similar on the COIL sets.

4.2. Visualization

One of the advantages of fully convolutional neural networks is that we can naturally visualize the inner activations (or features) and the trained weights (or filters) in a two-dimensional space [25]. Besides, we can monitor the learning process of DBC by drawing frequency hists of assignment scores. In addition, t-SNE can be applied to the embedded features to visualize the manifold structures in a low-dimensional space. Finally, we show some typical falsely categorized samples generated by our algorithm.

4.2.1. Visualization of the inner activations and learned filters

In Fig. 5, we visualize the inner activations of FCAE on the MNIST dataset with three digits: 1, 5, and 9. As shown in the figure, the activations in the feature layer are very sparse. Besides, the deconvolutional layer gradually recovers details of the pooled feature maps and finally gives a rough description of the original image. This indicates that FCAE can learn clustering-friendly features and keep the key information for image reconstruction.

Fig. 6 visualizes the learned filters of FCAE on the MNIST dataset. It is observed in [25] that the stacked convolutional auto-encoders trained on noisy inputs (30% binomial noise) and a max-pooling layer can learn localized biologically plausible filters. However, even without adding noise, the learned deconvolutional filters in our architectures are non-trivial Gabor-like filters which are visually the nicest shapes. This is due to the use of max-pooling and unpooling operations. As discussed in [25], the max-pooling layers are elegant way of enforcing sparse codes which are required to deal with the over-complete representations of convolutional architectures.

4.2.2. Monitoring the learning process

We use a frequency histogram of the soft assignment scores to monitor the learning process of DBC. The idea behind is that the histogram of the scores can provide statistical information about the learned cluster centers. When the scores with respect to a cluster center are distributed in two-side, it indicates that the cluster is well separated from other clusters. Fig. 7 shows the histogram of soft scores on the MNIST test dataset.[6] The scores are assigned to the first cluster (i.e., s_{i1}) at different learning epochs with varying epoch number t. At early epochs ($t = 0, 1$), most of the scores are around 0.1, which is a random guess probability since there are 10 clusters in total (i.e., $0.1 = \frac{1}{10}$). As the learning procedure goes on ($t = 2, 3, \ldots, 6$), some higher score samples are discriminatively boosted and their scores become larger than the others. Here the higher scores are gradually boosted from 0.1 to 0.5, while the lower scores are gradually decreased from 0.1 to 0.05. As a result, the cluster tends to "believe" in these higher score samples and consequently makes scores of the others to be smaller. Finally ($t = 50$), the scores assigned to the cluster become distributed in two-side. Samples with very high scores are thought to definitely belong to the first cluster, and the others with very small scores should belong to other clusters ($j \neq 1$). Here, the higher scores are approximately 0.8 and the lower scores are approximately 0.02, which indicates that the total score is around 1 (i.e., $0.8 + 9 \times 0.02$) by assuming that the clusters are uniformly distributed.

4.2.3. Embedding learned features in a low dimensional space

We visualize the distribution of the learned features in a two-dimensional space with t-SNE [33]. Fig. 8 shows the embedded features of the MNIST test dataset at different epochs. At the initial epoch, the features learned with FCAE are not very discriminative for clustering. As shown in Fig. 8(a), the features of digits 3, 5, and 8 are closely related. The same thing happened with digits 4, 7,

[6] Asubset of the MNIST dataset with 10000 samples.

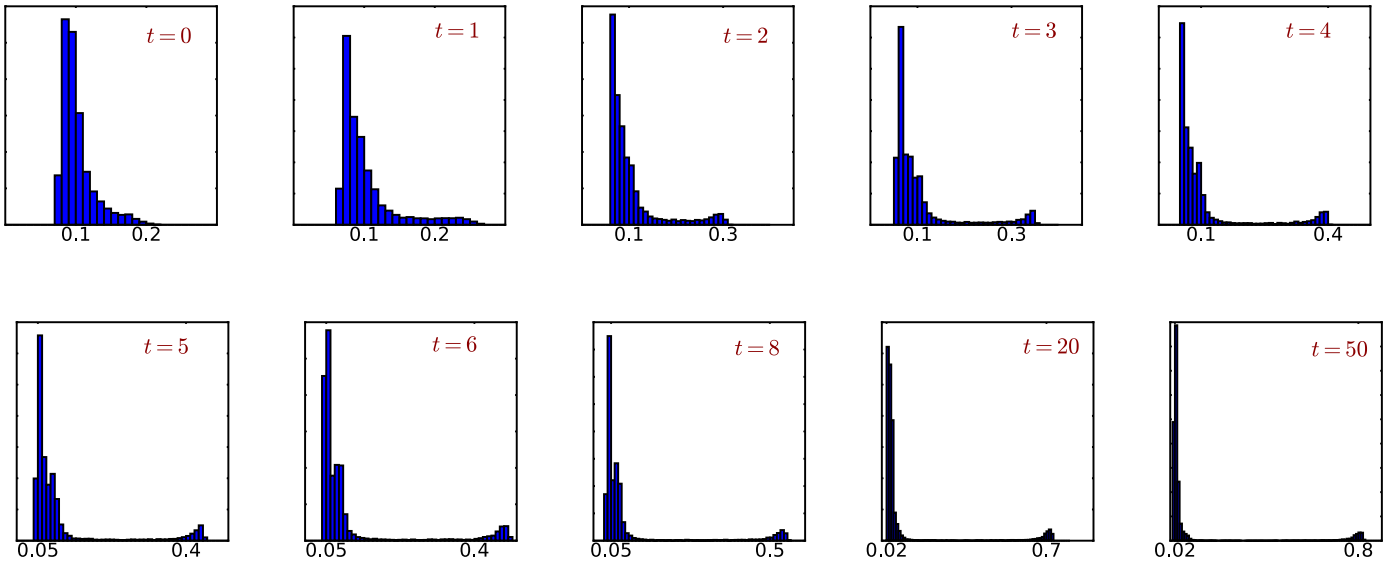
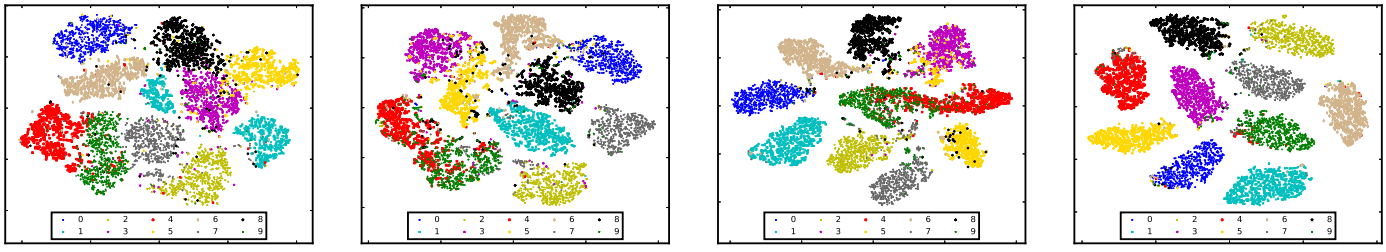


Fig. 7. Visualization of the soft score frequency hists with respect to the first cluster at different learning stages.



(a) epoch 0 (b) epoch 2 (c) epoch 5 (d) epoch 50

Fig. 8. Visualization of the embedded features in a two-dimensional space with *t*-SNE.

and 9. At the second epoch, the distribution of the learned features becomes much compact locally. Besides, the features of digit 7 become far away from those of digits 4 and 9. Similarly, the features of digit 8 get far away from those of digits 3 and 5. As the learning procedure goes on, the hardest digits (4 v.s. 9, 3 v.s. 5) for categorization are mostly well categorized after enough discriminative boosting epochs. The observation is consistent with the results showed in Section 4.2.2.

4.2.4. Visualization of falsely categorized examples

In Fig. 9, we show the top 100 falsely categorized examples whose maximum soft assignment scores are over 0.6. It can be observed that it is very hard to distinguish between some ground truth digits 4, 7 and 9 even with human experience. Lots of digits 7 are written with transverse lines in their middle space and would be thought to be ambiguous for the clustering algorithm. Besides, some ground truth images are themselves confusing, such as those showed with the gray background.

4.3. Discussions

In this section, we make some ablation studies on the learning process with respect to different boosting factors (α), different normalization methods (n_j) and different initialization models generated by FCAE.

4.3.1. Impact of the boosting factor α

Fig. 10(a) shows the ACC and NMI curves, where α equals to 1.5,2,4. With a small α ($\alpha = 1.5$), the learning process is very slow and takes very long time to terminate. On the contrary, when the factor is set to be very large ($\alpha = 4$), the learning process is very fast at the initial stages. However, this could result in falsely boosting some scores of the ambiguous samples. As a consequence, the model learned too much from some false information so the performance is not so satisfactory. With a moderate boosting factor ($\alpha = 2$), the ACC and NMI curves grow reasonably and progressively.

4.3.2. Impact of the balance normalization

In DEC [18], the authors pointed out that the balance normalization plays an important role in preventing large clusters from distorting the hidden feature space. To address this issue, we compare three normalization strategies: 1) the constant normalization for comparison, that is, $n_j = 1$, 2) the normalization by dividing the sum of the original soft assignment score per cluster, that is, $n_j = \sum_i s_{ij}$, which is adopted in DEC, and 3) the normalization by dividing the sum of the boosted soft assignment score per cluster, that is, $n_j = \sum_i s_{ij}^{\alpha}$. Fig. 10(b) presents the value curves of ACC and NMI against the epoch with these settings. Initially, the normalization does not affect ACC and NMI very much. However, the constant normalization can easily get stuck at early stages. The normalization by dividing $n_j = \sum_i s_{ij}$ has certain power of preventing the distortion. Our normalization strategy gives the best performance compared with the previous methods. This is because our normalization directly reflects the changes of the boosted scores.

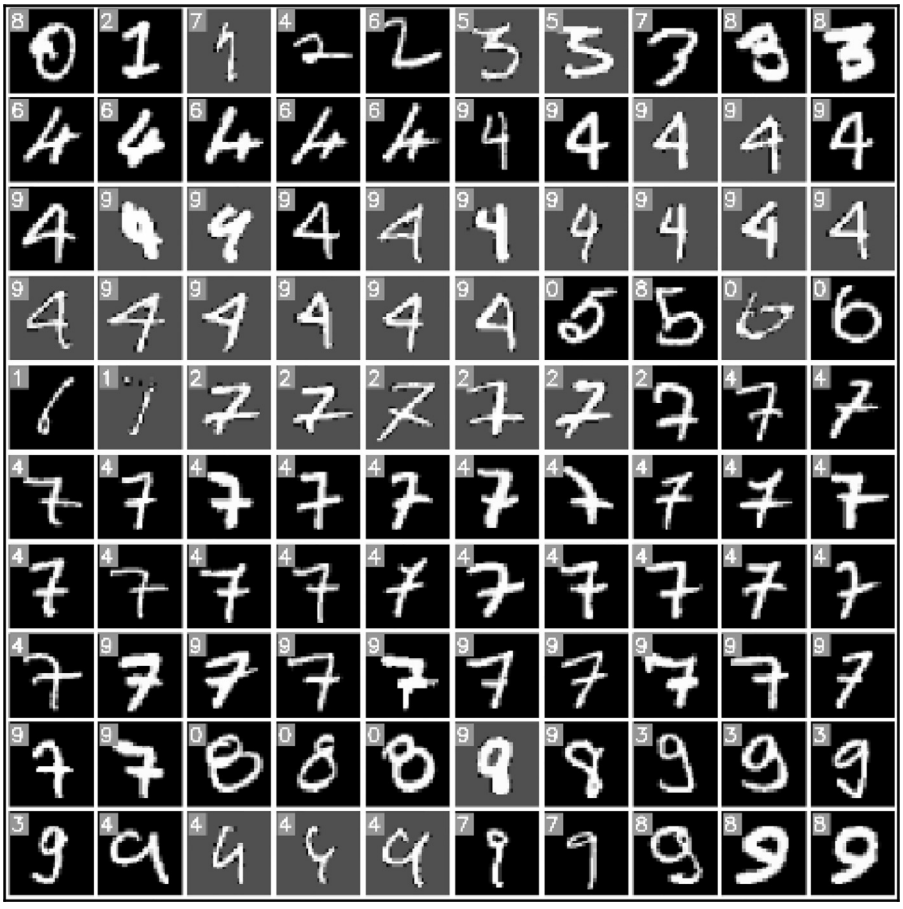
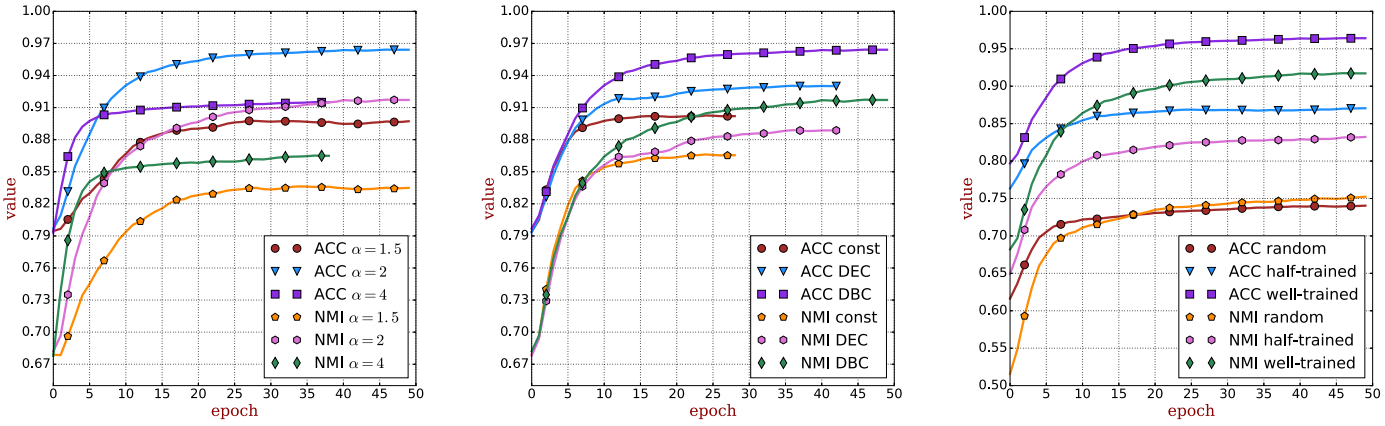


Fig. 9. Visualization of falsely categorized high score samples (top-100). The number in the top-left corner is the clustering label which is generated by the Hungarian algorithm.



(a) boosting factor (b) balance normalization (c) FCAE initialization

Fig. 10. Ablation studies with respect to the boosting factor α, the balance normalization factor n_j and the FCAE initialization models.

4.3.3. Impact of the FCAE initialization

To investigate the impact of the FCAE initialization on DBC, we compare the performance of DBC with three different initialization models: 1) the random initialization, 2) the initialization with a half-trained FCAE model, and 3) the initialization with a sufficiently trained FCAE model. The comparison results are shown in Fig. 10(c). As illustrated in the figure, DBC performs greatly based on all the models even when the initialization model is randomly distributed. However, if the FCAE model is not sufficiently trained, the resultant DBC model will be suboptimal.

5. Conclusions and future works

In this paper, we proposed FCAE and DBC to deal with image representation learning and image clustering, respectively. FCAE makes the first attempt to train convolutional auto-encoders in an end-to-end manner without greedy layer-wise pretraining. DBC reuses the FCAE features and learns deep image representations and cluster assignments jointly with a discriminatively boosting procedure. Benchmarks on several visual datasets show that DBC can achieve a superior performance than the analogous methods by

taking advantage of the following characteristics: 1) the convolutional architecture for 2-d/3-d structural features learning, 2) the batch-normalization for reducing internal covariate shift of deep neural networks, and 3) the discriminatively boosted learning procedure for features refinement. Besides, the visualization results show that the proposed learning algorithm can effectively implement the idea proposed in Section 3.2. Despite DBC has many merits, its clustering performance can be heavily affected by the cluster sparsity. In the future work, we may consider to solve this issue by using stacked denoising auto-encoders [22,36] and patch-wise training [26,36], which can be helpful for data augmentation. Besides, scaling the algorithm to deal with large-scale datasets such as the ImageNet dataset can also be considered in the future work.

Acknowledgments

This work was supported in part by NNSF of China under grants 91648205, 61627808, 61602483 and 61603389. We thank Xuanyang Xi, Shuguang Ding, Lu Qi and Yanfeng Lu for valuable discussions.

Appendix A. Derivation of (4)

We use the chain rule for the deduction. First, we set

$$u_{ij} = 1 + \frac{||z_i - \mu_j||^2}{\nu}. \tag{A.1}$$

Then it follows that

$$\frac{\partial u_{ij}}{\partial z_i} = \frac{2}{\nu}(z_i - \mu_j). \tag{A.2}$$

Now set

$$q_{ij} = u_{ij}^{-\frac{1+\nu}{2}}, \tag{A.3}$$

so

$$\begin{aligned}\frac{\partial q_{ij}}{\partial z_i} &= \frac{\partial q_{ij}}{\partial u_{ij}} \cdot \frac{\partial u_{ij}}{\partial z_i} \\ &= (-\frac{1+\nu}{2}) u_{ij}^{-\frac{3+\nu}{2}} \cdot \frac{2}{\nu}(z_i - \mu_j) \\ &= (-\frac{1+\nu}{\nu}) u_{ij}^{-1} q_{ij} \cdot (z_i - \mu_j).\end{aligned} \tag{A.4}$$

Further, let

$$s_{ij} = \frac{q_{ij}}{\sum_{j'} q_{ij'}}. \tag{A.5}$$

Then we have

$$\begin{aligned}\frac{\partial s_{ij}}{\partial z_i} &= \frac{\frac{\partial q_{ij}}{\partial z_i} \cdot \sum_{j'} q_{ij'} - q_{ij} \cdot \sum_{j'} \frac{\partial q_{ij'}}{\partial z_i}}{(\sum_{j'} q_{ij'})^2} \\ &= \frac{(-\frac{1+\nu}{\nu}) u_{ij}^{-1} q_{ij} \cdot (z_i - \mu_j) \cdot \sum_{j'} q_{ij'} - q_{ij} \cdot \sum_{j'} (-\frac{1+\nu}{\nu}) u_{ij'}^{-1} q_{ij'} \cdot (z_i - \mu_{j'})}{(\sum_{j'} q_{ij'})^2} \\ &= (-\frac{1+\nu}{\nu}) \frac{q_{ij}}{\sum_{j'} q_{ij'}} \frac{u_{ij}^{-1} \cdot (z_i - \mu_j) \cdot \sum_{j'} q_{ij'} - \sum_{j'} u_{ij'}^{-1} q_{ij'} \cdot (z_i - \mu_{j'})}{\sum_{j'} q_{ij'}} \\ &= (-\frac{1+\nu}{\nu}) s_{ij} (u_{ij}^{-1} \cdot (z_i - \mu_j) - \sum_{j'} u_{ij'}^{-1} s_{ij'} \cdot (z_i - \mu_{j'})).\end{aligned} \tag{A.6}$$

Combine the above expressions to get the required result

$$\frac{\partial L}{\partial z_i} = -\sum_j \frac{r_{ij}}{s_{ij}} \cdot \frac{\partial s_{ij}}{\partial z_i} \tag{A.7}$$

$$= \frac{1+\nu}{\nu} \sum_j \frac{r_{ij} - s_{ij}}{u_{ij}} (z_i - \mu_j)$$

$$= \frac{1+\nu}{\nu} \sum_j (r_{ij} - s_{ij}) \frac{z_i - \mu_j}{1 + ||z_i - \mu_j||^2/\nu} \tag{A.8}$$

Appendix B. Derivation of (5).

(5) can be derived similarly by exchanging μ and z in the above derivations of (4).

References

[1] J. Han, J. Pei, M. Kamber, Data Mining: Concepts and Techniques, Elsevier, 2011.
[2] P. Berkhin, A Survey of Clustering Data Mining Techniques, in: Grouping Multidimensional Data, 2006, pp. 25–71.
[3] J. Shi, J. Malik, Normalized cuts and image segmentation, IEEE Trans. Pattern Anal. Mach. Intell. 22 (8) (2000) 888–905.
[4] L. Wang, C. Pan, Robust level set image segmentation via a local correntropy-based k-means clustering, Pattern Recognit. 47 (5) (2014) 1917–1925.
[5] S. Choy, S. Lam, K. Yu, et al., Fuzzy model-based clustering and its application in image segmentation, Pattern Recognit. 68 (2017) 141–157.
[6] D. Tsai, C. Lin, Fuzzy c-means based clustering for linearly and nonlinearly separable data, Pattern Recognit. 44 (8) (2011) 1750–1760.
[7] M. Ester, H.P. Kriegel, J. Sander, X. Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, Knowl. Discov. Databases 96 (34) (1996) 226–231.
[8] A. Rodriguez, A. Laio, Clustering by fast search and find of density peaks, Science 344 (6191) (2014) 1492–1496.
[9] N. Ahmed, Recent review on image clustering, IET Image Process. 9 (11) (2015) 1020–1032.
[10] J. Yu, R. Hong, M. Wang, et al., Image clustering based on sparse patch alignment framework, Pattern Recognit. 47 (11) (2014) 3512–3519.
[11] C. Ding, T. Li, Adaptive dimension reduction using discriminant analysis and k-means clustering, in: Proceedings of the 24th International Conference on Machine learning, 2007, pp. 521–528.
[12] V. Sande, T. Gevers, C. Snoek, Evaluating color descriptors for object and scene recognition, IEEE Trans. Pattern Anal. Mach. Intell. 32 (9) (2010) 1582–1596.
[13] Z. Z. Guo, L. Zhang, D. Zhang, A completed modeling of local binary pattern operator for texture classification, IEEE Trans. Image Process. 19 (6) (2010) 1657–1663.
[14] S. Hong, J. Choi, J. Feyereisl, B. Han, L.S. Davis, Joint image clustering and labeling by matrix factorization, IEEE Trans. Pattern Anal. Mach. Intell. 38 (7) (2016) 1411–1424.
[15] M. Sampat, Z. Wang, S. Gupta, A. Bovik, M. Markey, Complex wavelet structural similarity: a new image similarity index, IEEE Trans. Image Process. 18 (1) (2009) 2385–2401.
[16] A. Krizhevsky, I. Sutskever, G. Hinton, Imagenet classification with deep convolutional neural networks, Adv. Neural Inf. Process. Syst. 24 (2012) 1097–1105.
[17] Y. Bengio, A. Courville, P. Vincent, Representation learning: a review and new perspectives, IEEE Trans. Pattern Anal. Mach. Intell. 35 (8) (2013) 1789–1828.
[18] J. Xie, R. Girshick, A. Farhadi, Unsupervised deep embedding for clustering analysis, in: Proceedings of the 33rd International Conference on Machine Learning, 2016, pp. 478–487.
[19] J. Yang, D. Parikh, D. Batra, Joint unsupervised learning of deep representations and image clusters, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016.
[20] H. Liu, M. Shao, S. Li, Y. Fu, Infinite ensemble clustering, in: Data Mining and Knowledge Discovery, 2017, pp. 1–32.
[21] G. Hinton, R. Salakhutdinov, Reducing the dimensionality of data with neural networks, Science 313 (5786) (2006) 504–507.
[22] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, P. Manzagol, Stacked denoising auto-encoders: learning useful representations in a deep network with a local denoising criterion, J. Mach. Learn. Res. 11 (December) (2010) 3371–3408.
[23] G. Hinton, S. Osindero, Y. Teh, A fast learning algorithm for deep belief nets, Neural Comput. 18 (7) (2006) 1527–1554.
[24] Y. Bengio, Learning deep architectures for AI, Found. Trends Mach. Learn. 2 (1) (2009) 1–127.
[25] J. Masci, U. Meier, D. Ciresan, J. Schmidhuber, Stacked convolutional auto-encoders for hierarchical feature extraction, in: Artificial Neural Networks and Machine Learning, 2011, pp. 52–59.
[26] H. Lee, R. Grosse, R. Ranganath, A. Ng, Unsupervised learning of hierarchical representations with convolutional deep belief networks, Commun ACM 54 (10) (2011) 95–103.
[27] F. Tian, B. Gao, Q. Cui, E. Chen, T. Liu, Learning deep representations for graph clustering, in: Proceedings of the 28th AAAI Conference on Artificial Intelligence, 2014, pp. 1293–1299.
[28] H. Larochelle, Y. Bengio, J. J. Louradour, et al., Exploring strategies for training deep neural networks, J. Mach. Learn. Res. 10 (2009) 1–40. Jan
[29] H. Noh, S. Hong, B. Han, Learning deconvolution network for semantic segmentation, in: Proceedings IEEE International Conference on Computer Vision, 2015, pp. 1520–1528.
[30] S. Ioffe, C. Szegedy, Batch normalization: accelerating deep network training by reducing internal covariate shift, in: Proceedings of the 32nd International Conference on International Conference on Machine Learning, 37, 2015, pp. 448–456.
[31] P. Huang, Y. Huang, W. Wang, L. Wang, Deep embedding network for clustering, in: International Conference on Pattern Recognition, 2014, pp. 1532–1537.
[32] C. Song, F. Liu, Y. Huang, et al., Auto-encoder based data clustering, in: Iberoamerican Congress on Pattern Recognition, 2013, pp. 117–124.

[33] L. Maaten, G. Hinton, Visualizing data using *t*-SNE, J. Mach. Learn. Res. 9 (2008) 2579–2605.
[34] H. Kuhn, The hungarian method for the assignment problem, in: 50 Years of Integer Programming 1958–2008, 2010, pp. 29–47.
[35] D. Cai, X. He, J. Han, Locally consistent concept factorization for document clustering, IEEE Trans. Knowl. Data Eng. 23 (6) (2011) 902–913.
[36] B. Du, W. Xiong, J. Wu, et al., Stacked convolutional denoising auto-encoders for feature representation, IEEE Trans. Cybern. 47 (4) (2017) 1017–1027.

Fengfu Li received the B.Sc. degree in mathematics and applied mathematics from University of Science and Technology of China, Hefei, China, in 2012 and the Ph.D. degree in machine learning and pattern recognition with Institute of Applied Mathematics, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing, China. His current research interests include clustering, unsupervised feature learning, manifold learning and deep learning.

Hong Qiao (SM'06) received the B.Eng. degree in hydraulics and control and the M.Eng. degree in robotics and automation from Xi'an Jiaotong University, Xian, China, and the Ph.D. degree in robotics from De Montfort University, Leicester, UK, in 1986, 1989, and 1995, respectively.

She was a University Research Fellow in De Montfort University, UK, a Research/Assistant Professor with City University of Hong Kong, Hong Kong, and a Lecturer with University of Manchester, Manchester, UK, from 1995 to 2004. She is currently a Professor with the State Key Lab of Management and Control for Complex Systems, Institute of Automation, Chinese Academy of Sciences, Beijing, China. Her current research interests include robotics, artificial intelligence, and machine learning.

She is currently an AdCom member of IEEE Robotics and Automation Society, an Associate Editor of the IEEE TRANSACTION ON CYBERNETICS and the IEEE TRANSACTION ON AUTOMATION SCIENCE AND ENGINEERING, and the Editor-in-Chief of Assembly Automation.

Bo Zhang (M'10) received the B.Sc. degree in mathematics from Shandong University, Jinan, China, the M.Sc. degree in mathematics from Xi'an Jiaotong University, Xian, China, and the Ph.D. degree in applied mathematics from the University of Strathclyde, Glasgow, UK, in 1983, 1985, and 1992, respectively.

After being a postdoc at Keele University, UK and a Research Fellow at Brunel University, UK, from 1992 to 1997, he joined Coventry University, Coventry, UK, in 1997, as a Senior Lecturer, where he was promoted to Reader in Applied Mathematics in 2000 and to Professor of Applied Mathematics in 2003. He is currently a Professor with Institute of Applied Mathematics, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing, China. His current research interests include direct and inverse scattering problems, radar and sonar imaging, machine learning, and data mining. He is currently an Associate Editor of the IEEE TRANSACTION ON CYBERNETICS, and Applicable Analysis.